Git & GitHub Daily Command Reference

A practical sequence for repository setup, branches, refresh, overwrite, commit, push, rename, recovery and daily workflow

Assumption: GitHub is the remote hosting service. Git manages the local repository; GitHub CLI (gh) is used below for GitHub-specific operations such as creating a repository and changing its visibility.

Important: Commands that discard local work are explicitly marked DESTRUCTIVE. Do not use them until you are sure you do not need the local changes.

0. One-time setup

```
git --version
gh --version
gh auth login
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

- git --version — verifies Git is installed.
- gh --version — verifies GitHub CLI is installed.
- gh auth login — authenticates the GitHub CLI.
- git config --global ... — sets your Git identity and default initial branch.

1. Create a repository on GitHub

Recommended when you want to create the GitHub repository from the command line:

```
gh repo create my-repo --private --description "My project description"
# or
gh repo create my-repo --public --description "My project description" --clone
```

- --private / --public — controls GitHub repository visibility.
- --description — sets the repository description.
- --clone — also clones the newly created repository locally.

If the repository already exists: Do not run git init on the local copy simply to connect it. Clone it, or add the remote with git remote add origin <URL>.

1.1 Change an existing GitHub repository from private to public (or vice versa)

```
gh repo edit OWNER/REPO --visibility public
gh repo edit OWNER/REPO --visibility private
```

Caution: Changing visibility can expose or hide repository contents. Check the repository history, issues, releases and any secrets before making a public repository.

1.2 Add or change the repository description

```
gh repo edit OWNER/REPO --description "New repository description"
# See current repository details:
gh repo view OWNER/REPO
```

2. Clone the repository to a local directory

```
git clone https://github.com/OWNER/REPO.git
cd REPO

# Clone into a specific local directory:
git clone https://github.com/OWNER/REPO.git my-local-folder
```

- git clone — downloads the repository, its Git history and the default branch, and creates the remote named origin.
- cd — moves into the working directory.

3. Checkout / pull a particular branch

First refresh the list of remote branches:

```
git fetch origin
git branch -a
```

If the remote branch already exists and you want a local tracking branch:

```
git switch --track origin/feature/my-branch
```

If you already have the local branch:

```
git switch feature/my-branch
git pull --ff-only origin feature/my-branch
```

3.1 Overwrite local tracked contents with the remote branch

```
git fetch origin
git switch feature/my-branch
git reset --hard origin/feature/my-branch
```

What this does: Makes tracked files in your local branch match the remote branch exactly at that commit. Uncommitted modifications to tracked files are discarded.

3.2 Make the entire working tree match the remote branch, including untracked files

```
git fetch origin
git switch feature/my-branch
git reset --hard origin/feature/my-branch
git clean -fd
```

DESTRUCTIVE: git clean -fd deletes untracked files and directories. Use git clean -fdx only if you also intentionally want to remove ignored files such as build artifacts, virtual environments, caches, etc.

3.3 Can Git ask for confirmation before overwriting each object?

There is no normal checkout/reset mode that interactively asks you to confirm every file/object before overwriting it. Safer inspection options are:

```
git status
git diff
git diff --name-only
git clean -nd      # preview untracked files/directories that clean -fd would delete
git clean -ndx     # preview including ignored files
```

- For selective restoration, use git restore interactively or restore specific paths rather than resetting the entire tree.

```
git restore --source origin/feature/my-branch -- path/to/file
git restore -p --source origin/feature/my-branch
```

Best practice: If you are unsure, create a temporary backup branch or copy before using reset --hard / clean.

3.a. Create a branch and write/overwrite its local contents

Create a new branch from your current commit:

```
git switch -c feature/my-branch
```

Create a new branch from a specific remote branch:

```
git fetch origin
git switch -c feature/my-branch --track origin/main
```

If the branch already exists on the remote and you want the local branch to be an exact copy of it:

```
git fetch origin
git switch -C feature/my-branch
git reset --hard origin/feature/my-branch
```

If the remote branch does not exist: After making local changes, push it with git push -u origin feature/my-branch.

4. Daily morning refresh — recommended routine

For normal collaborative work, avoid destructive resets. Start each day with:

```
git status
git switch main
git fetch origin
git pull --ff-only origin main
```

Or, if you work on a feature branch:

```
git status
git switch feature/my-branch
git fetch origin
git pull --ff-only origin feature/my-branch
```

Why fetch first?: git fetch updates your local knowledge of the remote without changing your working files. pull then integrates the remote branch into your current branch.

If you have local uncommitted changes: Do not blindly pull. Review git status first. Commit, stash, or otherwise handle the local work before integrating remote changes.

5. Commit and push new local changes

```
git status
git add .
git status
git commit -m "Describe the change"
git push
```

- git add . — stages new, modified and deleted files under the current directory.
- git status — lets you verify exactly what is staged before committing.
- git commit — creates a local commit.
- git push — sends your local commits to the configured upstream branch.

5.1 More controlled staging

```
git add path/to/file.py
git add path/to/directory/
git add -A
git add -p
```

- git add -A — stages all additions, modifications and deletions in the repository.
- git add -p — interactively selects individual hunks; useful when one file contains unrelated changes.

Recommendation: For a daily workflow, git add -A followed by git status is generally clearer than git add . because it makes the intent to stage repository-wide changes explicit.

6. Rename an existing file

```
git mv old_name.py new_name.py
git status
git commit -m "Rename old_name.py to new_name.py"
git push
```

Alternative: You can rename the file in Finder/Explorer or your IDE, then run git add -A. Git detects renames based on similarity; there is no separate rename object in Git.

7. Rename an existing folder

Git tracks files, not directories. Therefore a folder rename is really a rename/move of the files inside it.

```
git mv old-folder new-folder
git status
git add -A
git commit -m "Rename old-folder to new-folder"
git push
```

Important: An empty directory is not tracked by Git. If you need an otherwise-empty directory in a repository, a common convention is to place a .gitkeep file in it.

8. End-of-day — commit and push routine

Recommended routine when you want your work safely recorded on your branch:

```
git status
git diff
git add -A
git status
git commit -m "Describe today's completed work"
git push
```

If this is the first push of a newly created branch:

```
git push -u origin feature/my-branch
```

Do not commit blindly: Check git status before committing so you do not accidentally include secrets, credentials, large generated files, virtual environments or unrelated work.

8.1 If there is nothing to commit

```
git status
# If it says "nothing to commit, working tree clean":
git push
```

If there are no local commits to push: git push will normally report that everything is up to date.

9. Daily scenarios you are likely to encounter

9.1 See what changed

```
git status
git diff
git diff --cached
git diff HEAD
```

- git diff — unstaged changes.
- git diff --cached — staged changes.
- git diff HEAD — staged + unstaged changes compared with the last commit.

9.2 Temporarily put unfinished work aside

```
git stash push -m "WIP before sync"
git pull --ff-only
git stash pop
```

Safer variant: Use git stash list to see saved stashes. If stash pop causes conflicts, resolve them before continuing.

9.3 Pull causes a conflict

```
git status
# edit the conflicted files
git add <resolved-files>
git commit
git push
```

If you want to abandon the merge/rebase operation: The exact abort command depends on what Git started. git status will tell you whether you are in a merge or rebase; common commands are git merge --abort or git rebase --abort.

9.4 See branches

```
git branch
git branch -r
git branch -a
git branch -vv
```

9.5 Create, push and delete a branch

```
git switch -c feature/my-branch
git push -u origin feature/my-branch

# Delete local branch after it is merged:
git branch -d feature/my-branch

# Force-delete local branch:
git branch -D feature/my-branch

# Delete remote branch:
git push origin --delete feature/my-branch
```

Caution: Use -D only when you intentionally want to delete an unmerged local branch.

9.6 Undo local changes to one file

```
git restore path/to/file.py
```

DESTRUCTIVE: This discards the file's unstaged changes.

9.7 Unstage a file but keep its changes

```
git restore --staged path/to/file.py
```

9.8 Undo the last local commit but keep the changes

```
git reset --soft HEAD~1
```

Use case: Useful when you committed too early and want to amend/rework the changes. The changes remain staged.

9.9 Undo the last local commit and keep changes unstaged

```
git reset HEAD~1
```

9.10 Completely discard the last local commit and its changes

```
git reset --hard HEAD~1
```

DESTRUCTIVE: Only use when you are certain the commit and its changes are disposable.

9.11 Change the most recent commit message

```
git commit --amend -m "Correct commit message
```

If already pushed: Amending a pushed commit changes history. Avoid force-pushing shared branches unless your team explicitly permits it.

9.12 Accidentally committed a secret

```
# First: revoke/rotate the exposed credential immediately.
# Then remove it from the working tree and commit the correction.
git rm --cached path/to/secret-file
git commit -m "Remove secret from repository"
git push
```

Critical: Removing a secret from the latest commit does not erase it from Git history. For secrets that entered history, use a history-rewriting tool/process and rotate the credential. Never rely on .gitignore as a way to remove an already-committed secret.

9.13 Ignore files you do not want committed

```
# Create/edit .gitignore
git status
git add .gitignore
git commit -m "Add gitignore rules"
git push
```

Typical entries include .venv/, __pycache__/, .env, IDE metadata and build artifacts. Never put actual passwords or API keys in the repository.

9.14 Find who changed a line / inspect history

```
git log --oneline --decorate --graph --all
git log -- path/to/file.py
git blame path/to/file.py
git show <commit-id>
```

9.15 Check the remote URL

```
git remote -v
git remote show origin
```

9.16 Change the remote URL

```
git remote set-url origin https://github.com/OWNER/REPO.git
git remote -v
```

9.17 Synchronize all remote branch information

```
git fetch --all --prune
```

- --prune removes stale remote-tracking references for branches deleted from the remote.

9.18 Compare your branch with main

```
git fetch origin
git diff origin/main...HEAD
git log --oneline origin/main..HEAD
```

9.19 Update your feature branch with the latest main

Merge-based approach:

```
git switch feature/my-branch
git fetch origin
git merge origin/main
```

Rebase-based approach (cleaner linear history, but follow your team's policy):

```
git switch feature/my-branch
git fetch origin
git rebase origin/main
```

After rebase: If the branch was already pushed, updating the remote normally requires git push --force-with-lease. Prefer --force-with-lease over --force because it provides a safety check.

10. A simple branch strategy for personal GitHub projects

```
main
  ├── feature/llm-sql
  ├── feature/search-ranking
  ├── feature/data-profiling
  └── feature/mlops
```

- Keep main stable.
- Create one feature branch per meaningful feature or experiment.
- Work locally, commit frequently with meaningful messages, and push your feature branch regularly.
- Merge to main only when the feature is in a usable state.
- For a personal portfolio repository, you can also work directly on main, but feature branches make experimentation and recovery safer.

11. Recommended morning → work → evening sequence

Morning:

```
git switch <your-branch>
git fetch origin
git pull --ff-only origin <your-branch>
git status
```

During work:

```
git status
git add -A
git commit -m "Meaningful change"
git push
```

Evening / before closing shop:

```
git status
git diff
git add -A
git status
git commit -m "Complete <feature/task>"
git push
```

If you are not ready to make a meaningful commit, you can still create a WIP commit or use git stash. For a personal repository, frequent small commits are usually preferable to leaving an entire day's work uncommitted.

12. Quick command cheat sheet

Task	Command
Create GitHub repo	gh repo create NAME --private --description "..."
Clone	git clone <URL>
Refresh remote info	git fetch origin
Switch branch	git switch <branch>
Create branch	git switch -c <branch>
Pull safely	git pull --ff-only
Exact remote overwrite	git reset --hard origin/<branch>

Preview clean	git clean -nd
Stage everything	git add -A
Check changes	git status / git diff
Commit	git commit -m "message"
Push new branch	git push -u origin <branch>
Push existing branch	git push
Rename file/folder	git mv OLD NEW
Stash work	git stash push -m "WIP"
Restore one file	git restore <file>
Unstage one file	git restore --staged <file>
See history	git log --oneline --graph --all
See remote	git remote -v
Prune stale refs	git fetch --all --prune

13. The two routines worth memorizing

Start of day — safe refresh:

```
git status
git fetch origin
git pull --ff-only
```

End of day — record and publish work:

```
git status
git diff
git add -A
git status
git commit -m "Describe what changed"
git push
```

Golden rule: Before any destructive command such as git reset --hard, git clean -fd, git restore, or git reset --hard HEAD~1, run git status and make sure you understand exactly what will be lost.